

CompositeCurves

CompositeCurves is a ScriptableObject-driven piecewise curve system for Unity game tuning, progression, balancing, and procedural value generation.

Overview

The package provides:

- Piecewise curve assets through **CompositeCurveDefinition**
- Built-in presets: constant, linear, quadratic, cubic, sine, cosine, tangent, quadratic bezier, cubic bezier
- Per-segment variables for both presets and custom curves
- Inclusive or exclusive start/end bounds per segment
- A runtime API through **GetValue**, **Evaluate**, and **TryGetValue**
- An inspector with segment editing, validation, preview, sorting, and manual code generation
- Generated source for custom expressions in **CompositeCurveGenerated.Evaluator.g.cs**

Main Files

- Runtime asset: **CompositeCurveDefinition.cs**
- Runtime segment model: **CompositeCurveSegment.cs**
- Shared enums and variable model: **CompositeCurveTypes.cs**
- Editor inspector: **CompositeCurveDefinitionEditor.cs**
- Custom curve generator: **CompositeCurveCodeGenerator.cs**
- Contributor notes: **ARCHITECTURE.md**
- User-facing manual: **USER_MANUAL.md**

Authoring Summary

1. Create an asset through **Assets/Create/Composite Curves/Curve Definition**.
2. Add preset or custom segments in the inspector.
3. Adjust **Start X**, **End X**, bound inclusion, and variables for each segment.
4. For custom segments, write a single expression using **x** and your configured variable names.
5. Regenerate custom curves only when you want generated code to update.
6. Use **curve.GetValue(x)** from game code.

For full authoring instructions, see **USER_MANUAL.md**.

Runtime API

- **curve.GetValue(x)** returns a value for the given **x**
- **curve.Evaluate(x)** is an equivalent convenience entry point

- `curve.TryGetValue(x, out value)` reports whether the query produced a value from the current outside-range rules

Runtime example

```
using CompositeCurves;
using UnityEngine;

public sealed class CurveExample : MonoBehaviour
{
    [SerializeField] private CompositeCurveDefinition curve;
    [SerializeField] private float x;

    private void Update()
    {
        var y = curve != null ? curve.GetValue(x) : 0f;
        Debug.Log(y);
    }
}
```

Custom Curve Generation

- Custom expressions are not interpreted at runtime.
- The generator emits C# source into `CompositeCurveGenerated.Evaluator.g.cs`.
- Generation is manual only.
- Trigger it from the inspector button **Regenerate Custom Curves** or from **Tools/Composite Curves/Regenerate Generated Curves**.
- Common aliases such as `sin(x)`, `cos(x)`, `pow(a, b)`, and `pi` are translated to `Mathf.*`.

Inspector Notes

- New segments are created with their **Start X** set to the **End X** of the last existing segment.
- **Sort Segments** reorders by the actual domain range, not just the raw field order.
- The preview uses a persistent manual viewport.
- Drag to pan the preview.
- Scroll to zoom.
- **Shift+Scroll** zooms X only.
- **Alt+Scroll** zooms Y only.
- **Reset View** refits the preview manually.

Tests

- Edit mode tests live in `CompositeCurveRuntimeTests.cs`.

- Run them from the Unity Test Runner in Edit Mode.

Compatibility intent

- The code avoids package-specific dependencies beyond what is already present in the project.
- The implementation avoids very new language features and avoids deprecated Unity APIs where practical.
- “All Unity versions” cannot be guaranteed literally, but the code is being kept aligned with broadly supported editor/runtime APIs rather than newest-only features.